

NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



DATABASE SYSTEMS

REPORT

Topic: Personal Finance Management System

Instructor: Tran Hung

Student: Dinh Quoc Viet 11247150

Hanoi, 2026

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Project Objectives	2
1.3	System Requirements	2
1.4	Scope of the Project	3
1.5	Tools and Technologies	3
2	Database Design	3
2.1	Entity-Relationship (ER) Modeling	3
2.2	Relational Schema	4
2.3	Table Structures & Constraints	5
2.4	Relationship Design Rationale	5
3	Database Implementation	6
3.1	Sample Data Injection	6
3.2	Indexing Strategy	6
3.3	SQL Views for Data Abstraction	7
3.4	Stored Procedures for Transaction Safety	7
3.5	User-Defined Functions for Real-time Metrics	8
3.6	Automated Triggers for ACID Compliance	8
3.7	Backup & Recovery Protocols	8
4	Python Application Development	9
4.1	Database Connection Configuration	9
4.2	Main CLI Application Architecture	9
4.3	Validation and Data Integrity Mechanisms	10
4.4	Financial Analytics and Visualization	10
5	Database Security and Administration	11
5.1	Comprehensive Audit Trails	11
5.2	Access Control and Privileges	11
5.3	Performance Optimization	12
6	Conclusion and Recommendations	12
6.1	Conclusion	12
6.2	Recommendations for Future Work	12
7	References	13

1 Introduction

1.1 Project Overview

In an era where financial literacy and asset tracking are paramount, the Personal Finance Management System is a comprehensive, database-driven application engineered to assist individuals in systematically organizing, monitoring, and evaluating their financial activities. Unlike basic spreadsheet methods which are highly susceptible to human error and lack automated logic, this system delivers a secure, seamless workflow. It is designed to track multifaceted income streams, categorize daily expenditures, manage multiple bank accounts simultaneously, and analyze financial health through automated mathematical summaries. To ensure robust data tracking and strict auditing capabilities, the system integrates an automated balance history log that records every single account fluctuation with timestamp precision.

1.2 Project Objectives

The primary objective of this project is to architect a highly reliable and mathematically consistent system that empowers users to oversee their personal finances effectively. Specifically, the project aims to:

- Systematically track spending habits across customizable categories.
- Evaluate budget limits against real-time expenditures to prevent overspending.
- Optimize savings through data-driven reporting and visual analytics.
- Demonstrate the practical application of advanced relational database concepts.

This solution is built utilizing MySQL as the robust backend database engine, coupled with Python to provide an interactive, logic-driven Command-Line Interface (CLI) application.

1.3 System Requirements

To achieve the defined objectives, the developed system successfully fulfills the following core functional and non-functional requirements:

- **Data Management:** Secure, normalized storage and management of users, income streams, expenses, bank accounts, and spending categories.
- **Data Entry Workflow:** An interactive CLI for structured data entry, including categorized expenses and income logging with input validation.
- **Automated Auditing:** Continuous monitoring of bank accounts with a detailed, trigger-driven balance history that mathematically prevents silent data corruption.

- **Financial Evaluation:** Generation of dynamic monthly financial summaries and real-time budget status checks using SQL User-Defined Functions.
- **Analytical Reporting:** Visual reporting capabilities, specifically rendering Bar charts for cash flow comparisons and Pie charts for proportional spending analysis, ensuring intuitive financial insights.

1.4 Scope of the Project

This project emphasizes advanced relational database design, rigorous SQL implementation, and Python application development. The implemented architecture delves deeply into normalized schemas (up to 3NF), strategic indexing, complex aggregated views, scalar functions, parameterized stored procedures, and fully automated triggers. While web-based platforms or mobile interfaces are recognized as valuable for future expansion, the current scope strictly focuses on delivering a high-performance, feature-rich command-line application that acts as a robust proof-of-concept for the underlying database logic.

1.5 Tools and Technologies

The project relies on a modern, industry-standard technology stack:

- **Database Management System:** MySQL 8.x (Chosen for its robust ACID compliance and advanced Trigger/Procedure capabilities).
- **Database Administration Tool:** MySQL Workbench (Utilized for ERD modeling, script execution, and visual data inspection).
- **Programming Language:** Python 3.x (Chosen for its rapid prototyping capabilities and extensive data science libraries).
- **Python Libraries:** `mysql-connector-python` (For establishing secure database connections and executing queries), `matplotlib` (For rendering mathematical data visualizations).

2 Database Design

2.1 Entity-Relationship (ER) Modeling

The conceptual design of the database is the foundation of the system's logic, revolving around six core entities: `Users`, `ExpenseCategories`, `BankAccounts`, `Income`, `Expenses`, and `BalanceHistory`. The ER model enforces strict business rules: A single user can hold multiple bank accounts and record numerous transactions (both income and expenses), establishing 1-to-N relationships. Each expense must be mapped to exactly one specific category, allowing for precise aggregations. Furthermore, each bank account is

permanently tied to a historical ledger (**BalanceHistory**) that sequentially logs every balance adjustment, forming the backbone of the system’s auditing capability.

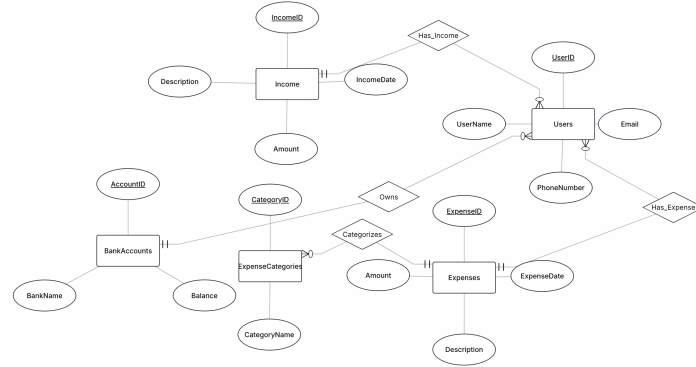


Figure 1: Entity-Relationship Diagram illustrating the core entities and their cardinalities

2.2 Relational Schema

Translating the conceptual ER model into a physical database, the relational schemas are strictly normalized to minimize redundancy and prevent update anomalies. The schema definitions are as follows:

- **Users** (UserID, UserName, Email, PhoneNumber)
- **ExpenseCategories** (CategoryID, CategoryName)
- **BankAccounts** (AccountID, UserID, BankName, Balance)
- **Income** (IncomeID, UserID, AccountID, Amount, IncomeDate, Description)
- **Expenses** (ExpenseID, UserID, CategoryID, AccountID, Amount, ExpenseDate, Description)
- **BalanceHistory** (HistoryID, AccountID, ChangeType, AmountChanged, Balance-Before, BalanceAfter, ChangedAt, ReferenceType, ReferenceID, Description)

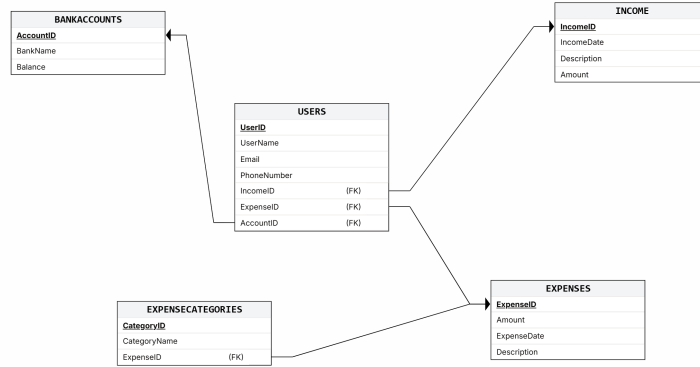


Figure 2: Relational Schema of the Database mapped from the ERD

2.3 Table Structures & Constraints

To guarantee absolute data integrity at the database level, tables are strictly governed by Primary Keys (PK) and Foreign Keys (FK). The implementation utilizes **ON DELETE CASCADE** rules strategically to ensure that if a user or an account is removed, all associated transactions are safely purged, preventing orphaned records. Furthermore, check constraints are applied mathematically to prevent negative transaction amounts, and **UNIQUE** constraints are enforced on user emails to definitively avoid duplicate account registrations.

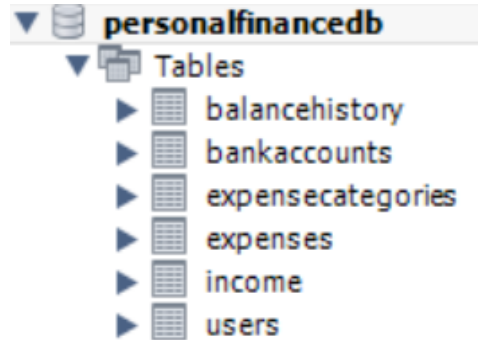


Figure 3: Main tables successfully created in the MySQL environment

2.4 Relationship Design Rationale

The relationship architecture explicitly mimics real-world financial workflows. Every financial transaction (Income or Expense) strictly requires an association with an existing, valid bank account. To provide full, banking-grade auditability, the **BalanceHistory** table establishes a mandatory 1-to-N relationship with **BankAccounts**. This ensures the system captures the exact timestamp, the specific transaction type (CREDIT/DEBIT),

and the mathematical delta of any balance modification, ensuring the current balance can always be verified against its historical ledger.

3 Database Implementation

3.1 Sample Data Injection

To rigorously validate system functionalities, trigger logic, and facilitate comprehensive report generation, mock data was systematically injected into the database. This data encompasses various user profiles, multiple bank accounts with initial balances, predefined spending categories, and a diverse array of income and expense transactions spanning several months.

	AccountID	UserID	BankName	Balance
▶	1	1	Vietcombank	12500000.00
	2	2	Techcombank	8400000.00
	3	3	MB Bank	25000000.00
	4	4	BIDV	5600000.00
	5	5	TPBank	18900000.00
	6	6	VietinBank	9200000.00
✱	NULL	NULL	NULL	NULL

Figure 4: Sample data injected into the BankAccounts table representing initial states

	ExpenseID	UserID	CategoryID	AccountID	Amount	ExpenseDate	Description
▶	1	1	1	1	500000.00	2026-04-05	Dinner with friends
	2	1	2	1	300000.00	2026-04-10	Gasoline
	3	1	3	1	2500000.00	2026-04-15	English Course
	4	1	4	1	400000.00	2026-04-20	Cinema tickets
	5	2	1	2	600000.00	2026-04-06	Groceries
	6	3	3	3	2000000.00	2026-04-12	Books
	7	4	4	4	150000.00	2026-04-16	Netflix
	8	5	5	5	1000000.00	2026-04-20	Electricity bill
	9	6	1	6	450000.00	2026-04-22	Cafe
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5: Sample data injected into the Expenses table for testing aggregations

3.2 Indexing Strategy

As the database scales, full table scans for financial reporting become computationally expensive. To optimize query response times, particularly for the **GROUP BY** and **WHERE**

clauses used in generating financial reports, non-clustered B-Tree indexes were established. These indexes target high-frequency search and join columns, specifically `UserID`, `AccountID`, `CategoryID`, and chronological transaction dates (`IncomeDate`, `ExpenseDate`).

Comment	Index_comment	Visible	Expression
		YES	NULL
		YES	NULL
		YES	NULL
		YES	NULL

Figure 6: Index definitions implemented to optimize query execution plans

3.3 SQL Views for Data Abstraction

To adhere to the principle of separation of concerns, three primary SQL Views were constructed. These views abstract complex `JOIN` operations and mathematical aggregations, providing a simplified, virtual table layer for the Python backend to query directly:

- **MonthlySummaryView**: Aggregates total income, calculates total expense, and computes the net mathematical difference on a month-by-month basis per user.
- **CategorySpendingView**: Groups and sums up expenditures by their respective categories, serving as the direct data source for the application’s Pie Chart.
- **BalanceHistoryView**: Formats and displays chronological account balance alterations, joining with the `Users` table to provide human-readable audit logs.

3.4 Stored Procedures for Transaction Safety

To standardize data entry and prevent SQL injection vulnerabilities from the application layer, the business logic for all transaction insertions was strictly encapsulated into Stored Procedures, namely `AddIncomeProc` and `AddExpenseProc`. These procedures process incoming parameters, execute the `INSERT` statements, and safely commit the transactions to the database as a single atomic operation.

	IncomeID	UserID	AccountID	Amount	IncomeDate	Description
▶	7	1	1	800000.00	2026-04-25	Test Procedure
	6	6	6	10000000.00	2026-04-05	Salary April
	5	5	5	20000000.00	2026-04-02	Salary April
	4	4	4	8000000.00	2026-04-15	Freelance
	3	3	3	5000000.00	2026-04-10	Project Bonus
	2	2	2	12000000.00	2026-04-05	Salary April
	1	1	1	15000000.00	2026-04-01	Salary April
✱	NULL	NULL	NULL	NULL	NULL	NULL

Figure 7: Execution of the AddIncomeProc Stored Procedure

3.5 User-Defined Functions for Real-time Metrics

Custom scalar SQL functions were engineered to compute real-time metrics dynamically at the database level, thereby significantly reducing the computational load and data transfer overhead on the Python application. Notable functions include `fn_budget_status`, which evaluates a user's total monthly spending against a dynamic parameter to return status warnings (e.g., 'Over Budget'), and `fn_current_balance`, which instantly retrieves up-to-date, verified account funds.

3.6 Automated Triggers for ACID Compliance

Triggers act as the immutable, automated core of the system's accounting logic, guaranteeing that the database state remains perfectly consistent regardless of how data is inserted:

- **Initialization:** Automatically records an 'Opening Balance' log upon the creation of any new bank account.
- **Auto-Calculation:** Dynamically increments or decrements the `Balance` field in the `BankAccounts` table immediately following an `Income` or `Expense` insertion.
- **Auditing:** Simultaneously inserts a detailed, immutable log into the `BalanceHistory` table reflecting the exact mathematical state of the account before and after the transaction, ensuring zero data discrepancy.

3.7 Backup & Recovery Protocols

To safeguard against catastrophic data loss and ensure system resilience, a complete backup script utilizing the MySQL `mysqldump` utility was implemented. Crucially, the inclusion of the `--routines` and `--triggers` flags ensured that all advanced database objects (Procedures, Functions, and Triggers) were successfully archived alongside the table data in the backup file.

```
C:\Program Files\MySQL\MySQL Server 8.4\bin>mysqldump -u root -p --routines --triggers PersonalFinanceDB > C:\Users\Public\PersonalFinanceDB_backup_final.sql
Enter password: *****
C:\Program Files\MySQL\MySQL Server 8.4\bin>
```

Figure 8: Database backup created successfully using the mysqldump utility

4 Python Application Development

4.1 Database Connection Configuration

The CLI application interfaces seamlessly with the MySQL engine via the `mysql-connector-python` library. The connection parameters are securely managed and isolated within the `db.py` module. Adhering to strict database security protocols, the application deliberately avoids utilizing the default `root` account. Instead, it connects via a dedicated, restricted user role (`finance_admin`), ensuring the principle of least privilege is maintained.

4.2 Main CLI Application Architecture

The user interaction layer, located in `main.py`, provides a robust, loop-driven menu interface offering 18 distinct, highly granular functionalities. Users can easily navigate through numerical options to register new accounts, securely log financial transactions, review chronological history logs, evaluate their budget status against custom limits, and command the system to generate analytical visual charts.

```
===== PERSONAL FINANCE MANAGEMENT SYSTEM =====
1. Add Income
2. Add Expense
3. View Monthly Summary
4. View Category-wise Spending
5. Check Budget Status
6. View Users
7. View Categories
8. View Bank Accounts
9. View Income History
10. View Expense History
11. Report: Income vs Expense Chart
12. Report: Category Spending Pie Chart
13. Add User
14. Add Bank Account
15. View Accounts by User
16. View Balance History
17. View Current Balance
0. Exit
Enter your choice: █
```

Figure 9: The comprehensive Command-Line Interface (CLI) menu of the system

4.3 Validation and Data Integrity Mechanisms

Validation is rigorously enforced at two distinct architectural layers to prevent system failure. On the application side (Python), comprehensive `try-except` blocks catch `ValueError` exceptions to prevent the script from crashing due to invalid user data types (e.g., inputting text where a float is expected). Furthermore, logical queries verify the existence and ownership of an `AccountID` before allowing the application to call a transaction procedure. On the database side, strict foreign key constraints and the aforementioned triggers reject any logically invalid operations, such as attempting to process an expense if the account lacks sufficient funds.

4.4 Financial Analytics and Visualization

By integrating the `matplotlib` library, the Python application transcends simple text-based data retrieval, providing users with immediate, visual financial analytics that aid in rapid decision-making:

- **Bar Chart (Option 11):** Contrasts the aggregated Total Income directly against Total Expenses for a user-specified year. This provides an immediate, highly visual overview of macro financial growth and cash flow efficiency.
- **Pie Chart (Option 12):** Breaks down monthly spending proportionally by category. This visual highlights exactly where the user's capital is being allocated, easily identifying areas for potential budget cuts.

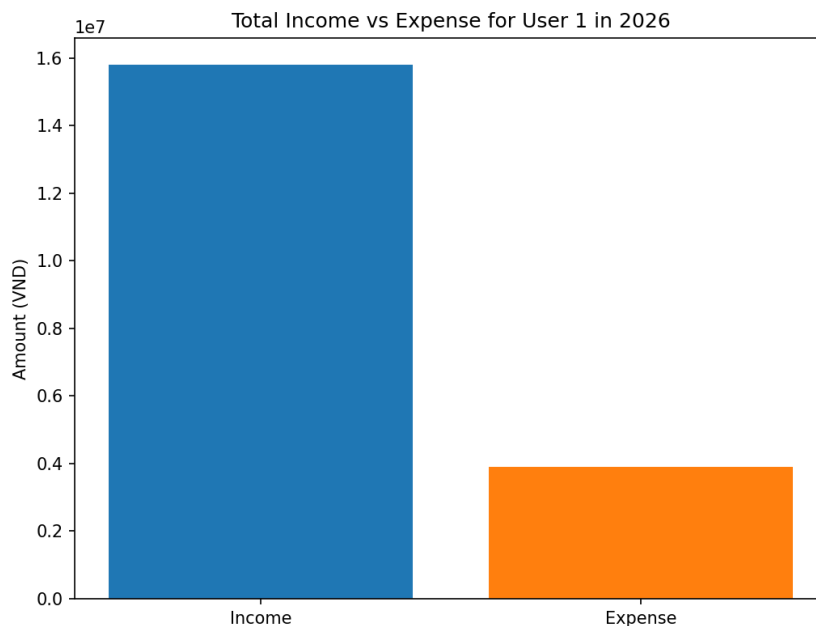


Figure 10: Income versus Expense Bar Chart rendered via matplotlib

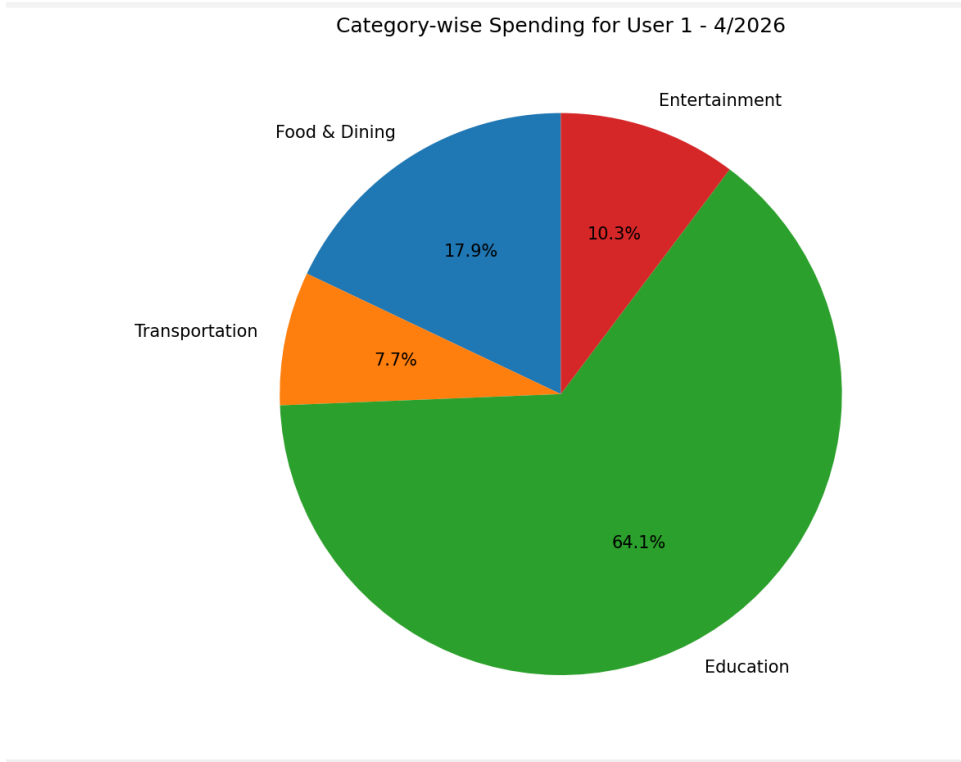


Figure 11: Category-wise Spending Pie Chart illustrating proportional expenditure

5 Database Security and Administration

5.1 Comprehensive Audit Trails

The design and implementation of the `BalanceHistory` table represent a critical security and administrative triumph for the system. Driven entirely by backend triggers, it guarantees a 100% transparent, tamper-proof historical record of exactly where, when, and how funds were moved. This ensures the system operates with the reliability and auditability of a professional banking ledger.

5.2 Access Control and Privileges

Application-level security is strictly enforced by abandoning the highly vulnerable `root` user for daily operations. A specialized, purpose-built account, `finance_admin` (secured by a complex password), was established via Data Control Language (DCL) commands. This account is granted specific privileges only on the `PersonalFinanceDB` schema, drastically minimizing potential security vulnerabilities in the event of an application compromise.

5.3 Performance Optimization

As transaction volumes inherently grow over time in a financial system, full table scans become computationally expensive and degrade user experience. The strategic deployment of B-Tree indexes on foreign keys and chronological date columns significantly reduced the query execution time required for the database to aggregate the `MonthlySummaryView` and supply data for the `matplotlib` rendering engine.

6 Conclusion and Recommendations

6.1 Conclusion

The Personal Finance Management System successfully and comprehensively demonstrates the powerful integration of advanced relational database architecture with a functional Python-based application. By thoroughly leveraging MySQL's complex procedural features—specifically Views, Stored Procedures, User-Defined Functions, and automated Triggers—the system successfully automates tedious accounting calculations, ensures mathematically strict data integrity, and maintains a flawless, tamper-proof audit trail of balance histories. The development of the Python CLI conclusively proves that the underlying database architecture is highly functional and accessible, successfully enabling users to log complex transactions, monitor custom budgets, and render insightful visual financial reports efficiently.

6.2 Recommendations for Future Work

While the current command-line interface provides a remarkably fast, stable, and effective operational environment, future iterations of this project hold significant potential for expansion:

- **UI/UX Modernization:** Developing a Graphical User Interface (GUI) utilizing PyQt, or transitioning to a Web Application using frameworks like Flask or Django, to provide a more intuitive user experience.
- **Machine Learning Integration:** Implementing predictive analytics algorithms utilizing Python's `scikit-learn` library to forecast upcoming monthly expenses and cash flow shortages based on historical data trends.
- **Data Exportability:** Integrating features to export complex financial reports and charts into portable formats such as PDF or Excel files for user convenience.
- **Open Banking APIs:** Establishing secure API connections for real-time bank statement syncing, eliminating the need for manual transaction entry.

7 References

- MySQL 8.4 Reference Manual. Oracle Corporation.
- Python 3 Documentation. Python Software Foundation.
- `mysql-connector-python` Official Documentation. Oracle Corporation.
- `matplotlib` User Guide. The Matplotlib Development Team.
- Dinh Quoc Viet (2026). *Personal Finance Management System Source Code*. GitHub Repository: <https://github.com/vietdinh2209/personal-finance-management-system>